

Лабораторная работа №6

Разработка веб-приложения на Spring Boot. Основы тестирования

Цель работы

Изучить основы тестирования в Java и Spring Boot-приложениях: научиться писать модульные тесты, проверять поведение отдельных компонентов приложения, изолировать зависимости с помощью мок-объектов, а также познакомиться с базовыми подходами к тестированию сервисного слоя и веб-приложений.

Теоретическая часть

Тестирование — это важная часть разработки программного обеспечения, которая позволяет убедиться, что приложение работает корректно и продолжает работать правильно после внесения изменений.

В контексте Spring Boot-приложения тесты особенно полезны, потому что приложение состоит из нескольких слоев:

- `controller` — принимает HTTP-запросы;
- `service` — содержит бизнес-логику;
- `repository` — работает с данными;
- `security` — отвечает за безопасность и доступ.

На практике разные части приложения удобно проверять разными типами тестов:

- **unit tests** — тестируют отдельный класс или метод изолированно от остальных частей системы;
- **integration tests** — проверяют совместную работу нескольких компонентов;
- **web tests** — проверяют работу HTTP-слоя и контроллеров.

JUnit

JUnit — это основной фреймворк для написания тестов в Java. Он предоставляет:

- аннотацию `@Test` для объявления тестового метода;
- аннотации жизненного цикла, например `@BeforeEach` и `@AfterEach`;
- методы проверок (`assertEquals`, `assertTrue`, `assertFalse`, `assertNotNull`, `assertThrows` и другие);
- возможность группировать, запускать и организовывать тесты.

JUnit рассматривает тест как обычный Java-метод, который подготавливает данные, вызывает проверяемый код и сравнивает фактический результат с ожидаемым.

Mockito

Mockito — библиотека для создания мок-объектов. Mock — это специальный объект-заглушка, который имитирует поведение реальной зависимости.

Mock-объекты полезны, когда мы хотим протестировать один класс отдельно от остальных. Например, если мы тестируем `UserService`, то нам не нужно обращаться к реальной базе данных. Вместо реального `UserRepository` можно использовать mock-объект, который будет возвращать заранее подготовленные значения.

Mockito предоставляет:

- `@Mock` — создание mock-объекта;
- `@InjectMocks` — автоматическое внедрение mock-объектов в тестируемый класс;
- `when(...).thenReturn(...)` — описание поведения mock-объекта;
- `verify(...)` — проверка того, что нужный метод действительно был вызван;
- `spy(...)` — частичное мокирование реального объекта.

Главная идея Mockito: тестируем только логику проверяемого класса, а его зависимости заменяем контролируруемыми mock-объектами.

Ход выполнения работы

Часть 0. Подготовка

1. Используйте проект `spring-lab3-notifications`, расширенный в лабораторных №4 и №5.
2. Убедитесь, что приложение запускается и содержит сущности `User` и `Notification`, сервисы, контроллеры и репозитории.
3. Откройте `pom.xml` и проверьте наличие тестовой зависимости Spring Boot.

Как правило, в проекте уже присутствует:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
```

Эта зависимость подключает базовый набор инструментов для тестирования Spring Boot-приложения, включая JUnit и другие вспомогательные библиотеки для тестов. Spring Boot отдельно выделяет тестирование как часть стандартного стека приложения. (docs.spring.io)

Если Mockito не подтягивается автоматически или вы хотите показать его отдельно, можно добавить зависимость явно:

```
<dependency>
  <groupId>org.mockito</groupId>
  <artifactId>mockito-core</artifactId>
  <scope>test</scope>
</dependency>
```

Mockito рекомендует подключать именно `mockito-core` как основную библиотеку для работы с моки-объектами. (site.mockito.org)

Часть 1. Первая структура unit-теста на JUnit

Рассмотрим самый простой пример теста без Spring Boot-контекста.

Создайте класс `SimpleTest` в `src/test/java`:

```
package org.example;

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

public class SimpleTest {

    @Test
    void shouldAddNumbers() {
        int result = 2 + 3;
        assertEquals(5, result);
    }
}
```

Что здесь происходит:

- `@Test` помечает метод как тестовый;
- внутри теста выполняется проверяемая логика;
- `assertEquals(expected, actual)` сравнивает ожидаемое и фактическое значение.

JUnit предоставляет стандартный набор аннотаций, assertions, обработку исключений, параметризованные тесты и другие механизмы для написания тестов. (junit.org)

Часть 2. Основные возможности JUnit для unit-тестов

2.1. Проверка значений

Пример:

```
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

public class AssertionsTest {

    @Test
    void shouldCheckAssertions() {
        String value = "Spring";
    }
}
```

```

        assertEquals("Spring", value);
        assertNotNull(value);
        assertTrue(value.startsWith("Sp"));
        assertFalse(value.isEmpty());
    }
}

```

Часто используемые методы:

- `assertEquals()` — проверяет равенство значений;
- `assertNotNull()` — проверяет, что объект не `null`;
- `assertTrue()` — проверяет истинность выражения;
- `assertFalse()` — проверяет ложность выражения.

2.2. Проверка исключений

```

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertThrows;

public class ExceptionTest {

    @Test
    void shouldThrowException() {
        assertThrows(ArithmeticException.class, () -> {
            int result = 10 / 0;
        });
    }
}

```

`assertThrows()` позволяет проверить, что код действительно выбрасывает ожидаемое исключение.

2.3. Методы жизненного цикла

```

import org.junit.jupiter.api.*;

public class LifecycleTest {

    @BeforeEach
    void setUp() {
        System.out.println("Подготовка перед каждым тестом");
    }

    @AfterEach
    void tearDown() {
        System.out.println("Завершение после каждого теста");
    }
}

```

```

@Test
void firstTest() {
    System.out.println("Первый тест");
}

@Test
void secondTest() {
    System.out.println("Второй тест");
}
}

```

- `@BeforeEach` — выполняется перед каждым тестом;
- `@AfterEach` — выполняется после каждого теста.

Это удобно для подготовки общих данных, создания тестируемых объектов и очистки состояния.

Часть 3. Что такое unit-тест в нашем приложении

Для нашего приложения системы уведомлений unit-тесты удобнее всего писать для сервисного слоя, потому что именно там находится основная бизнес-логика.

Хорошие кандидаты для unit-тестирования:

- `UserService`;
- `NotificationService`;
- `AuthService` из лабораторной по Spring Security;
- возможные utility-классы и mapper-классы.

Плохие кандидаты для чистого unit-теста:

- `JpaRepository`, потому что он тесно связан с базой данных;
- сложные Spring-конфигурации, если цель — именно unit-тест.

Идея проста:

- тестируем один класс;
- реальные зависимости заменяем mock-объектами;
- проверяем только собственную логику этого класса.

Часть 4. Первый unit-тест с Mockito для `UserService`

Предположим, у нас есть такой сервис:

```

@Service
@RequiredArgsConstructor
public class UserService {

    private final UserRepository userRepository;

    public User createUser(UserDto request) {
        User user = new User();
    }
}

```

```

        user.setName(request.getName());
        user.setEmail(request.getEmail());
        user.setPhone(request.getPhone());
        user.setDeviceToken(request.getDeviceToken());
        user.setTelegramChatId(request.getTelegramChatId());
        user.setCreatedAt(LocalDate.now());

        return userRepository.save(user);
    }
}

```

Теперь напишем для него unit-тест:

```

package org.example.service;

import org.example.model.dto.UserDto;
import org.example.model.entity.User;
import org.example.repository.UserRepository;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertNotNull;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.when;

@ExtendWith(MockitoExtension.class)
class UserServiceTest {

    @Mock
    private UserRepository userRepository;

    @InjectMocks
    private UserService userService;

    @Test
    void shouldCreateUser() {
        UserDto dto = UserDto.builder()
            .name("Иван Иванов")
            .email("ivan@example.com")
            .phone("+79990001122")
            .deviceToken("device-token-123")
            .telegramChatId("123456789")
            .build();

        User savedUser = new User();
        savedUser.setName(dto.getName());
    }
}

```

```

        savedUser.setEmail(dto.getEmail());
        savedUser.setPhone(dto.getPhone());
        savedUser.setDeviceToken(dto.getDeviceToken());
        savedUser.setTelegramChatId(dto.getTelegramChatId());

        when(userRepository.save(any(User.class))).thenReturn(savedUser);

        User result = userService.createUser(dto);

        assertNotNull(result);
        assertEquals("Иван Иванов", result.getName());
        assertEquals("ivan@example.com", result.getEmail());
    }
}

```

Что здесь важно:

- `@ExtendWith(MockitoExtension.class)` подключает Mockito к JUnit 5;
- `@Mock` создает mock-объект `userRepository`;
- `@InjectMocks` создает объект `userService` и внедряет в него mock-репозиторий;
- `when(...).thenReturn(...)` задает поведение mock-объекта;
- assertions проверяют результат метода.

Mockito поддерживает создание mock-объектов, stubbing через `when()`, проверки через `verify()`, а также аннотации `@Mock`, `@Spy` и `@InjectMocks`. (site.mockito.org)

Часть 5. Проверка вызова метода репозитория через `verify()`

Иногда недостаточно проверить только результат. Нужно убедиться, что зависимость была вызвана нужным образом.

```

package org.example.service;

import org.example.model.dto.UserDto;
import org.example.model.entity.User;
import org.example.repository.UserRepository;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.verify;
import static org.mockito.Mockito.when;

@ExtendWith(MockitoExtension.class)
class UserServiceVerifyTest {

    @Mock

```

```

private UserRepository userRepository;

@InjectMocks
private UserService userService;

@Test
void shouldCallSaveOnRepository() {
    UserDto dto = UserDto.builder()
        .name("Иван")
        .email("ivan@example.com")
        .build();

    when(userRepository.save(any(User.class))).thenReturn(new User());

    userService.createUser(dto);

    verify(userRepository).save(any(User.class));
}
}

```

`verify()` позволяет проверить, что метод зависимости действительно был вызван. Mockito отдельно выделяет `verify()` как один из основных механизмов тестирования взаимодействий. (site.mockito.org)

Часть 6. Тестирование NotificationService

Теперь рассмотрим пример, где сервис зависит сразу от двух репозиториев.

Упрощенная логика NotificationService:

```

public Notification createNotification(NotificationDto request) {
    User user =
userRepository.findById(request.getRecipientId()).orElseThrow();

    Notification notification = new Notification();
    notification.setTitle(request.getTitle());
    notification.setMessage(request.getMessage());
    notification.setChannel(request.getChannel());
    notification.setStatus(NotificationStatus.CREATED);
    notification.setCreatedAt(LocalDateTime.now());
    notification.setRecipient(user);

    return notificationRepository.save(notification);
}

```

Тест:


```

package org.example.service;

import org.example.model.dto.NotificationDto;
import org.example.model.entity.Notification;
import org.example.model.entity.User;
import org.example.model.enums.NotificationChannel;
import org.example.repository.NotificationRepository;
import org.example.repository.UserRepository;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.util.Optional;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertNotNull;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.when;

@ExtendWith(MockitoExtension.class)
class NotificationServiceTest {

    @Mock
    private NotificationRepository notificationRepository;

    @Mock
    private UserRepository userRepository;

    @InjectMocks
    private NotificationService notificationService;

    @Test
    void shouldCreateNotification() {
        User user = new User();
        user.setId(1L);
        user.setEmail("ivan@example.com");

        NotificationDto dto = NotificationDto.builder()
            .title("Напоминание")
            .message("Завтра пара по Spring")
            .channel(NotificationChannel.EMAIL)
            .recipientId(1L)
            .build();

        Notification savedNotification = new Notification();
        savedNotification.setTitle(dto.getTitle());
        savedNotification.setMessage(dto.getMessage());
        savedNotification.setChannel(dto.getChannel());
    }

```

```

        savedNotification.setRecipient(user);

        when(userRepository.findById(1L)).thenReturn(Optional.of(user));

        when(notificationRepository.save(any(Notification.class))).thenReturn(savedNotification);

        Notification result = notificationService.createNotification(dto);

        assertNotNull(result);
        assertEquals("Напоминание", result.getTitle());
        assertEquals(NotificationChannel.EMAIL, result.getChannel());
        assertEquals(user, result.getRecipient());
    }
}

```

В этом примере mock-объекты позволяют полностью изолировать `NotificationService` от реальной базы данных.

Часть 7. Проверка исключений в сервисах

В реальном приложении важно тестировать не только успешные сценарии, но и ошибки.

Например, в `NotificationService` пользователь может не найтись по `recipientId`.

```

package org.example.service;

import org.example.model.dto.NotificationDto;
import org.example.model.enums.NotificationChannel;
import org.example.repository.NotificationRepository;
import org.example.repository.UserRepository;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.util.Optional;

import static org.junit.jupiter.api.Assertions.assertThrows;
import static org.mockito.Mockito.when;

@ExtendWith(MockitoExtension.class)
class NotificationServiceExceptionTest {

    @Mock
    private NotificationRepository notificationRepository;

    @Mock
    private UserRepository userRepository;

```

```

@InjectMocks
private NotificationService notificationService;

@Test
void shouldThrowExceptionWhenUserNotFound() {
    NotificationDto dto = NotificationDto.builder()
        .title("Напоминание")
        .message("Сообщение")
        .channel(NotificationChannel.EMAIL)
        .recipientId(99L)
        .build();

    when(userRepository.findById(99L)).thenReturn(Optional.empty());

    assertThrows(RuntimeException.class, () ->
        notificationService.createNotification(dto));
}
}

```

Такой тест проверяет, что сервис корректно обрабатывает ошибочный сценарий.

Часть 8. Mock, Stub и Spy

Чтобы не путаться в терминах, полезно различать несколько понятий.

Mock

Mock — объект, который имитирует поведение зависимости и позволяет проверять взаимодействие с ней.

Пример:

```

@Mock
private UserRepository userRepository;

```

Stub

Stub — это заранее заданное поведение mock-объекта.

Пример:

```

when(userRepository.findById(1L)).thenReturn(Optional.of(user));

```

То есть mock — это сам объект-заглушка, а stub — это описание того, что именно он должен вернуть или сделать.

Spy

Spy — объект, который оборачивает реальный объект. У него можно использовать настоящую логику, но при этом частично переопределять отдельные методы и проверять вызовы.

Пример:

```
List<String> list = new ArrayList<>();
List<String> spyList = spy(list);

spyList.add("Spring");
verify(spyList).add("Spring");
```

Mockito поддерживает `spy()` и `@Spy` как механизм частичного мокирования. (site.mockito.org)

Часть 9. Когда не нужно использовать мок-объекты

Использовать мок-объекты удобно, но не всегда правильно.

Не стоит:

- мокировать простые value-объекты;
- мокировать весь мир вокруг тестируемого класса без необходимости;
- подменять тестами реальную бизнес-логику так, что тест перестает проверять что-либо полезное.

Mockito отдельно рекомендует не мокировать всё подряд и использовать mocks осознанно. (site.mockito.org)

Часть 10. Базовое тестирование контроллера

Для самого первого знакомства можно показать простой web-тест через `@WebMvcTest`.

Предположим, у нас есть `UserController`.

```
package org.example.controller;

import org.example.service.UserService;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.test.web.servlet.MockMvc;

import static
    org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static
    org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;
```

```

@WebMvcTest(UserController.class)
class UserControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private UserService userService;

    @Test
    void shouldReturnOk() throws Exception {
        mockMvc.perform(get("/users/all"))
            .andExpect(status().isOk());
    }
}

```

Что показывает этот пример:

- `@WebMvcTest` поднимает только веб-слой, а не всё приложение;
- `MockMvc` позволяет отправлять HTTP-запросы без реального запуска сервера;
- `@MockBean` подменяет зависимость контроллера mock-объектом внутри Spring-контекста.

Spring Boot предоставляет отдельные возможности для тестирования приложений, включая web-тесты и другие специализированные режимы. (docs.spring.io)

Часть 11. Разница между unit-тестом и интеграционным тестом

Unit-тест

- тестирует один класс;
- не использует реальную базу данных;
- не требует поднятия всего Spring-контекста;
- обычно выполняется быстро.

Примеры в этой лабораторной:

- `UserServiceTest`;
- `NotificationServiceTest`.

Интеграционный тест

- проверяет совместную работу нескольких частей системы;
- может использовать Spring-контекст;
- может работать с тестовой базой данных;
- обычно выполняется медленнее.

Часть 12. Проверка работы тестов в IntelliJ IDEA

1. Создайте папку `src/test/java`, если она еще не создана.
2. Добавьте тестовые классы в тот же пакетный путь, что и основные классы.
3. Запустите тест через зеленую стрелку рядом с методом или классом.
4. Убедитесь, что тесты выполняются успешно.

5. Попробуйте намеренно изменить ожидаемое значение в одном из assertions, чтобы увидеть падение теста.

Самостоятельные задания

1. Напишите unit-тест для метода `getUserById()` в `UserService`.
2. Напишите unit-тест для метода `deleteUser()` и проверьте, что вызывается `userRepository.delete(...)`.
3. Реализуйте тест для метода `getNotificationById()`.
4. Добавьте негативный тест для случая, когда уведомление не найдено.
5. Создайте отдельный класс тестов для `AuthService` из лабораторной по Spring Security.
6. Используйте `verify(..., times(1))` для явной проверки количества вызовов.
7. Попробуйте применить `spy()` к простому списку и проверьте вызов метода `add()`.
8. Напишите простой `@WebMvcTest` для `NotificationController`.

Контрольные вопросы

1. Что такое unit-тест?
2. Чем unit-тест отличается от интеграционного теста?
3. Что делает аннотация `@Test`?
4. Для чего используются `assertEquals()` и `assertThrows()`?
5. Что делают `@BeforeEach` и `@AfterEach`?
6. Что такое mock-объект?
7. Чем mock отличается от stub?
8. Для чего используется Mockito?
9. Что делают `@Mock` и `@InjectMocks`?
10. Зачем нужен `verify()`?
11. Что такое `spy()`?
12. Почему для unit-тестов сервисов удобнее использовать mock-репозитории?
13. Что делает `@WebMvcTest`?
14. Для чего нужен `MockMvc`?
15. Почему unit-тесты должны быть изолированными и быстрыми?